Appello 11 luglio 2017



Politecnico di Milano

Anno accademico 2015-2016

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Cugola	☐ Mottola	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. La consegna di un compito gravemente insufficiente comporta l'impossibilità a partecipare all'appello successivo.
6. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1

Si consideri la seguente classe Java `Digrafo<T>`. Essa descrive un ADT che rappresenta un grafo orientato, i cui nodi hanno un'etichetta del tipo generico T. Un nodo particolare, detto radice, è distinto dagli altri. Tutti i nodi del grafo sono raggiungibili dalla radice tramite un cammino orientato

I nodi del digrafo sono oggetti di una classe immutabile, detta `Nodo<T>`, non meglio precisata. Tale classe contiene solo l'etichetta del nodo, mentre non contiene riferimenti ai nodi ad esso collegati (che sono mantenuti separatamente in una opportuna struttura dati all'interno dalla classe `Digrafo<T>`).

```
public class Digrafo<T> {
//OVERVIEW: Un digrafo connesso con un nodo speciale detto radice, tale che per ogni nodo
//c'e' un cammino dalla radice al nodo.
//Da ogni nodo escono esattamente 0 oppure R>0 archi, dove R (detto grado del grafo) e'
//una costante fissata al momento della costruzione del grafo.
//Ogni nodo e' etichettato con un valore di tipo T.

  public final int R;

   //@ensures (* costruisce un digrafo con radice n, con R==grado,
   //senza altri nodi ne' archi, lancia eccezione se grado e' scorretto o n nullo *)
   public Digrafo(Nodo<T> n, int grado)  throws NonValidoException; ;

   //observers:

   //@ensures (* \result e' la radice del digrafo *)
   public /*@ pure @*/ Nodo<T> radice();

   //@ensures (* \result e' il numero di nodi del digrafo *);
   public /*@ pure @*/ int size();

   //@ensures (* \result vale true se, e solo se, il nodo n e' nel digrafo *);
   public /*@ pure @*/ boolean contains( Nodo<T> n);


   //@ensures (* \result e' l'insieme che contiene gli R nodi del grafo collegati
   // con un arco uscente dal nodo n, o l'insieme vuoto se non vi sono archi uscenti*);
   public /*@ pure @*/ Set<Nodo<T>> out(Nodo<T> n);

   //@ensures (* \result e' l'insieme di tutti e soli i nodi
   //@collegati con un arco entrante nel nodo n*);
   public /*@ pure @*/ Set<Nodo<T>> in(Nodo<T> n);

   //@ensures (* restituisce true se esiste cammino  dal nodo n al nodo h
   public /*@ pure @*/ boolean raggiungibile(Nodo<T> n, Nodo<T> h)

   //modifiers:

   //@ensures (* elimina tutti gli archi e tutti i nodi del grafo, salvo la radice *)
   public  void  cancellaGrafo()

   //@ensures (* Collega ciascuno dei nodi dell'insieme insNodi, supposto non vuoto,
   //@ con un un arco uscente dal nodo n.
   //@ I nodi in insNodi, che non sono gi\'a presenti nel grafo, sono ad esso aggiunti.
   //@ Lancia eccezione, senza modificare il grafo, se n non esiste nel digrafo oppure
   //@ se l'aggiunta degli archi renderebbe non valido il grado.
   public void collega(Nodo<T> n,  Set<Nodo<T>> insNodi) throws NonValidoException;

}
```

Domanda a

Utilizzando la notazione JML, descrivere la specifica del costruttore e del metodo `collega`. Si ipotizzi che le specifiche di tutti gli altri metodi della classe siano già state definite.

Soluzione:

```
//@ensures n!=null && grado>0 && R == grado && radice()==n && contains(n) &&
//@        in(n).size()==0 && out(n).size()==0 && size()==1;
//@signals (NonValidoException e) n==null || grado<=0;
public Digrafo(Nodo<T> n, int grado) ;

 //@ensures n!=null && insNodi!=null &&
 //@        \old(contains(n) && insNodi.size()==R && out(n).size()==0) &&
 //@        (\forall Nodo<T> h; insNodi.contains(h); out(n).contains(h) &&
 //@        in(h).contains(n)) && out(n).size()==R &&
 //@       (\forall Nodo<T> h;; contains(h) <==> \old(contains(h) || insNodi.contains(h)) );
 //@
 //@signals (NonValidoException e) (n==null || insNodi==null ||
 //@        !\old(contains(n) && insNodi.size()==R && out(n).size()==0)) &&
 //@       (\forall Nodo<T> h;; \old(contains(h) <==> contains(h));
 //@
 public void collega(Nodo<T> n,  Set<Nodo<T>> insNodi) throws NonValidoException;
```

Domanda b

Si definiscano, tramite un opportuno inviarante astratto, le proprietà descritte nell'overview, che sono qui ricordate: Un digrafo connesso con un nodo speciale detto radice, tale che per ogni nodo c'e' un cammino dalla radice al nodo. Da ogni nodo escono esattamente 0 oppure `R>0` archi.

Soluzione:

L'invariante asserisce che R è il grado del grafo e che il grafo è connesso.

```
//@public invariant size()>=1 &&
//@  (\forall Nodo<T> h; this.contains(h); (out(h).size()==0 || out(h).size()==R)) &&
//@           raggiungibile(h));
```

Domanda c

Si consideri ora la specifica della seguente classe:

```
public class DigrafoFlessibile<T>  extends Digrafo<T>  {

\\OVERVIEW: Un Digrafo in cui e' possibile eliminare tutti gli archi uscenti da un nodo

  //@ensures (* Elimina tutti gli archi uscenti da n
  //@ Se il nodo n non esiste nel grafo, allora il metodo lancia eccezione*);
   public void elimina(Nodo<T> n) throws NonValidoException;

}
```

La classe `DigrafoFlessibile<T>` può ereditare da `Digrafo<T>` in base al principio di sostituzione di Liskov? Motivare la risposta.

Soluzione:

No, eliminando un nodo il grafo potrebbe non essere piu' connesso, violando la regola delle proprietà.

Domanda d

Si consideri la seguente implementazione della classe `DigrafoFlessibile<T>`, che utilizza una classe Arco, definita nello stesso package. Si noti che la classe `DigrafoFlessibile<T>` e' definita con gli identici metodi di `Digrafo<T>`, con in piu' il solo metodo `elimina`.

```
public class DigrafoFlessibile<T> {
   //rep:
   private ArrayList<Nodo<T>> nodi;
   private ArrayList<Arco> archi;

   public DigrafoFlessibile<T>(Nodo<T> n, int grado) {
    if (n== null || grado<=0) throw new NonValidoException();
    R=grado;
    nodi = new ArrayList<Nodo<T>();
    archi = new ArrayList<Arco>();
    nodi.add(n);
    }
   ...implementazione di tutti i metodi dell'interfaccia.
}

class Arco {
   Nodo<T> sorgente;
   Nodo>T> destinazione;
}
```

L'implementazione memorizza in `nodi` tutti i nodi del grafo, con la radice in prima posizione, e in `archi`, tutti e soli gli archi, disposti in un ordine non precisato.

Si scriva l'invariante di rappresentazione della classe `DigrafoFlessibile<T>`.

Soluzione:

L'invariante richiede che gli archi colleghino solo nodi memorizzati in `nodi` e che per ogni nodo in `nodi` ci siano esattamente 0 oppure R archi di cui il nodo è il primo elemento.

```
//@private invariant nodi!=null && archi!= null && nodi.size()>=1 &&
//@  (\forall Arco a; archi.contains(a); nodi.contains(a.sorgente) && nodi.contains(a.destinaz
//@  (\forall Nodi<T> n; nodi.contains(n);
//@           (\num_of Arco a; archi.contains(a); a.sorgente==n) ==R ||
//@           (\num_of Arco a; archi.contains(a); a.sorgente==n) ==0;
```

Esercizio 2

Si consideri la seguente classe Java:

```
public class Coppia {
  private int myX, myY;
  public Coppia() {
    myX=0; myY=0;
  }
  public void set(int x, int y) {
    if( (x+y)>=0 ) {
      myX = x;
      myY = y;
    }
  }
  public int getSomma() {
    return myX+myY;
  }
}
```

Domanda a

In presenza di thread multipli é corretto affermare che il metodo `getSomma` restituisce sempre un valore maggiore o uguale a 0? Motivare la propria risposta.

Soluzione

La coppia di assegnamenti nel metodo `set` non viene eseguita in maniera atomica, due thread diversi potrebbero quindi invocare il metodo con due coppie di valori che prese come tali superano il test `x+y>=0` ma danno luogo ad una somma negativa se i valori vengono "mischiati". Ad esempio, il primo thread potrebbe invocare `set(100,-10)`. Subito dopo l'assegnamento `myX=x`, che in questo caso assegna `100` a `myX`, il secondo thread potrebbe ottenere il controllo, invocando `set(-100,200)`. Anche questo thread passerebbe il controllo eseguendo gli assegnamenti previsti per cui avremmo: `myX==-100` e `myY==200`. A questo punto il primo thread potrebbe riprendere l'esecuzione, impostando `myY=-10`. Al termine del processo avremmo: `myX==-100, myY==-10`. La somma dei due valori risulta ovviamente negativa.

Domanda b

In caso di risposta negativa alla precedente domanda, come bisogna modificare il codice per garantire che il metodo `getSomma` restituisca sempre un valore maggiore o uguale a 0?

Soluzione

É sufficiente rendere `synchronized` **entrambi** i metodi `set` e `getSomma`.

Domanda c

Modificare la classe `Coppia` aggiungendo un metodo `aspettaZero()` che sospende il thread chiamante fin tanto che la somma `myX+myY` non sia pari a 0 (se necessario, oltre ad aggiungere il nuovo metodo, si modifichino i metodi esistenti).

Soluzione

```
public class Coppia {
  private int myX, myY;
  public Coppia() {
    myX=0; myY=0;
  }
  public synchronized void set(int x, int y) {
    if( (x+y)>=0 ) {
      myX = x;
      myY = y;
      if( (x+y)==0 ) notifyAll();
    }
```

```
  }
  public synchronized int getSomma() {
    return myX+myY;
  }
  public synchronized void aspettaZero() {
    while( (x+y)!=0 ) wait();
  }
}
```

Esercizio 3

Si considerino le seguenti definizioni di classi:

```
abstract public class Animal {
   abstract public void greeting();
}
public class Cat extends Animal {
   public void greeting() {
      System.out.println("Meow!");
   }
}
public class Dog extends Animal {
   public void greeting() {
      System.out.println("Woof!");
   }
   public void greeting(Dog another) {
      System.out.println("Wooooooooof!");
   }
}
public class BigDog extends Dog {
   public void greeting() {
      System.out.println("Woow!");
   }
   public void greeting(Dog another) {
      System.out.println("Wooooowwwww!");
   }
}
```

Si consideri ora la seguente esecuzione di un ipotetico metodo, e si descrivano *i)* eventuali errori segnalati dal compilatore, e *ii)* una volta rimosse le righe che eventualmente generano errrori di compilazione, quale sia l'output prodotto a video. Per ciascuna riga prodotta a video, si spieghi inoltre, in base alla gerarchia delle classi e ai tipi statici e dinamici delle diverse variabili riferimento, il motivo per cui viene eseguito uno specifico metodo tra quelli disponibili.

```
Animal animal1 = new Cat();
animal1.greeting();
Animal animal2 = new Dog();
animal2.greeting();
Animal animal3 = new BigDog();
animal3.greeting();
Animal animal4 = new Animal();
animal4.greeting();

BigDog bigDog1 = new BigDog();
Dog dog2 = (Dog)animal2;
BigDog bigDog2 = (BigDog)animal3;
Dog dog3 = (Dog)animal3;

dog2.greeting(dog3);
dog3.greeting(dog2);
dog2.greeting(bigDog2);
bigDog2.greeting(dog2);
bigDog2.greeting(bigDog1);
```

Soluzione:

La riga `Animal animal4 = new Animal()` genera un errore di compilazione: la classe `Animal` è astratta. Di conseguenza, anche la riga successiva `animal4.greeting()` deve essere rimossa perchè il codice possa compilare.

Le prime tre invocazioni di `greeting()` sono banali: eseguono il codice del metodo corrispondente al tipo dinamico, nell'ordine `Cat`, `Dog`, e `BigDog`. Le ultime cinque invocazioni di `greeting(parametro)` stampano rispettivamente:

```
Wooooooooof!
Wooooowwwww!
Wooooooooof!
Wooooowwwww!
Wooooowwwww!
```

Di queste, il caso degno di nota è il secondo: sebbene `dog3` abbia un tipo statico `Dog`, il riferimento `animal3` ha un tipo dinamico `BigDog`, per cui le regole di ereditarietà e polimorfismo determinano che sia il codice di quest'ultimo ad essere seguito. Notare che in tutti i casi il parametro di ingresso viene ignorato.

Esercizio 4

Si consideri il metodo seguente e si generi un insieme di casi di test che soddisfi il criterio di copertura delle diramazioni (branch coverage).

```
public int foo(int a, int b) {
  int temp;
  if ( a==b ) {
    return 0;
  } else {
    if ( a>b ) {
      b=a*a;
      temp=0;
    } else {
      temp=1;
    }
    while ( a<b ) {
      temp=temp+1;
      a=a+2;
    }
  }
  return temp;
}
```

Soluzione:
Sono sufficienti tre casi di test: (1,1), (2,1) e (1,2).